

Group 7
Sadia, Avneet , Shifath
Data Base Management System
Prof. Dr. Syed Ahmad Chan Bukhari
12/08/2025

Phase: 5

SECTION 1: SYSTEM OVERVIEW & PROJECT EVOLUTION

1.1 Project Purpose and Real-World Application

The Mathnasium Tutoring Database is a comprehensive relational database management system designed to streamline and automate the day-to-day operations of Mathnasium learning centers. In the real world, tutoring centers like Mathnasium face significant operational challenges including student enrollment tracking, tutor scheduling, attendance monitoring, payment processing, and progress reporting. Without a centralized database system, these operations become error-prone, time-consuming, and difficult to scale across multiple locations.

Our database solution addresses these critical business needs by providing:

- **Centralized Student Management:** Track student information, enrollments, and academic progress across multiple subjects
- **Efficient Tutor Scheduling:** Manage tutor qualifications, availability, and session assignments
- **Automated Payment Processing:** Record and track all financial transactions linked to package plans
- **Progress Tracking:** Enable tutors to document student strengths, weaknesses, and goals after each session
- **Multi-Location Support:** Scale operations across different Mathnasium centers while maintaining data integrity

1.2 Scope and Coverage

Our database comprehensively models **10 core entities** that represent every major operational component of a tutoring center:

Strong Entities:

1. **LOCATION** - Physical addresses and timezone information for each center
2. **CENTER** - Individual Mathnasium learning center branches
3. **TUTOR** - Teaching staff with hiring dates and contact information
4. **SUBJECT** - Academic subjects offered (Elementary Math, Algebra, etc.)
5. **STUDENT** - Enrolled learners with demographic details

6. **PARENT_GUARDIAN** - Contact persons responsible for students
7. **PACKAGE_PLAN** - Tutoring membership packages available for purchase
8. **PAYMENT** - Financial transaction records

Weak Entity: 9. **PROGRESS_REPORT** - Dependent on SESSION, tracks student performance

Associative Entities: 10. **ENROLLMENT** - Resolves many-to-many relationship between STUDENT and SUBJECT 11. **SESSION** - Scheduled tutoring meetings 12. **SESSION_ATTENDANCE** - Links students to sessions with check-in/out times 13. **TUTOR_SUBJECT** - Tracks which subjects each tutor is qualified to teach 14. **STUDENT_PARENT** - Links students to their guardians

1.3 How Our Design Evolved: Phase 1 → Phase 5

Phase 1: Problem Definition & Requirements

We began by clearly defining the operational problems faced by a Mathnasium tutoring center like managing students, tutors, subjects, scheduling, payments, and progress reporting. This phase allowed us to identify the main business processes and the entities needed to support them.

Phase 2: Draft ER/EER Modeling

In this phase, we created the initial draft of our ER/EER diagram.
This draft captured:

- Preliminary entities and attributes
- Initial relationships and cardinalities
- Early identification of weak entities and M:N relationships

Phase 2 focused on translating the conceptual understanding into a diagram, but the model was still flexible and subject to improvement in the next phase.

Phase 3: Refined ER/EER Model & Complete Data Dictionary (Earned 15/15)

In Phase 3, we refined the ER/EER diagram using feedback and course material, then built a full Data Dictionary.

This included:

- Finalizing all entities, attributes, and keys
- Normalizing the design to Third Normal Form (3NF)

- Defining participation constraints and identifying all associative entities
- Clarifying business rules and domain constraints
- Documenting every attribute with data types and descriptions

Our refined model and Data Dictionary received a perfect score of 15/15, confirming the correctness and completeness of our design.

Phase 4: SQL Implementation & Database Construction

We translated the refined logical design into a working MySQL database.

Our Phase 4 work included:

- Writing CREATE TABLE statements with PK, FK, NOT NULL, UNIQUE, and CHECK constraints
- Inserting realistic sample data
- Testing SELECT queries to verify relationships
- Adjusting data types or constraints when necessary for accuracy and integrity

This phase produced the complete physical database structure used for Phase 5 simulations.

Phase 5: Final Integration, Demonstration & Validation (Current Phase)

Phase 5 brings all previous work together into a fully functional demonstration.

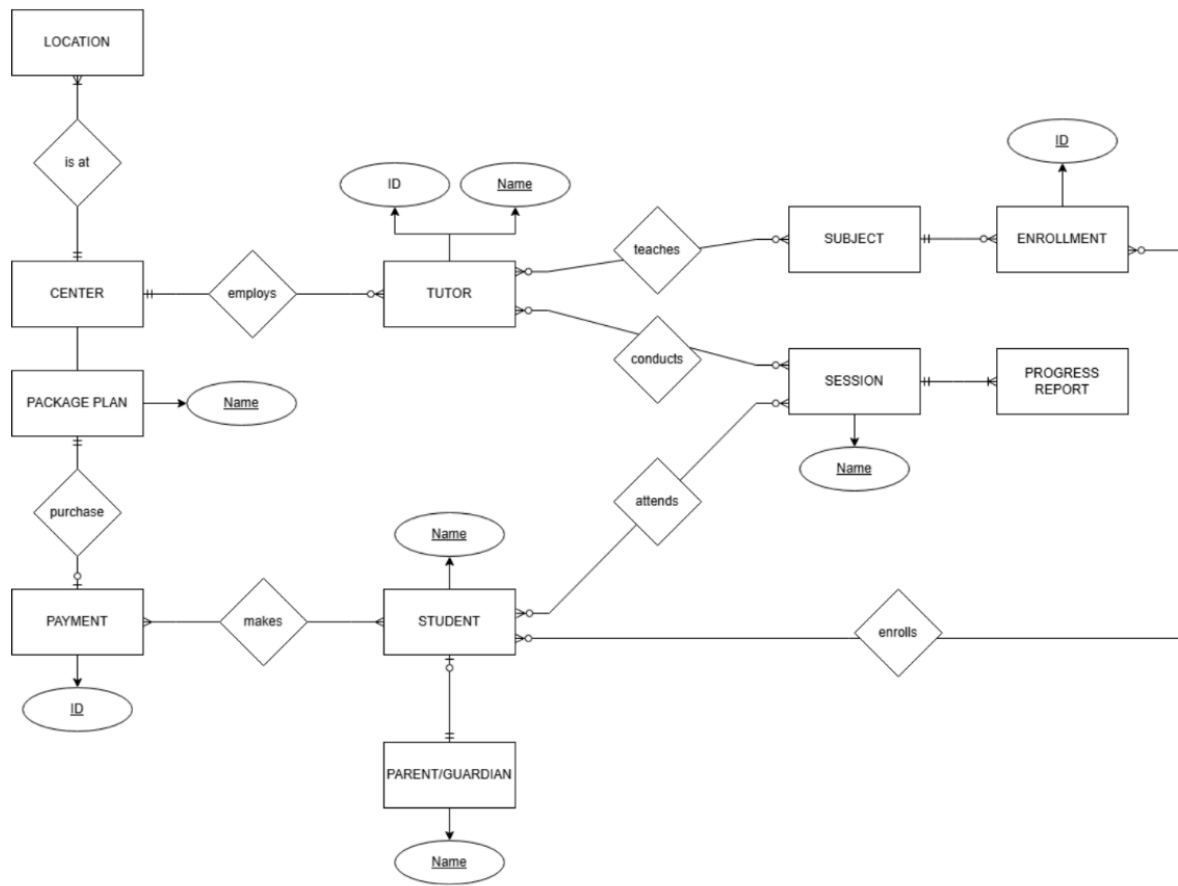
We show how the database operates in realistic scenarios by:

- Running multi-table JOIN queries
- Simulating real-world interactions (student registration, session scheduling, payment recording, etc.)
- Validating constraints and testing error rejection
- Providing reports or analytical outputs
- Describing future enhancements for scalability and performance

1.4 Entity-Relationship Diagram

Below is our finalized ER diagram showing all entities, relationships, cardinalities, and attributes:

Figure 1.1: Complete ER Diagram for Mathnasium Database



1.5 Key Design Decisions and Their Justification

Decision 1: Separating LOCATION and CENTER

Rationale: A single physical location might host multiple centers or sub-branches in the future. This separation provides scalability and maintains location data consistency.

Decision 2: Making PROGRESS_REPORT a Weak Entity

Rationale: Progress reports cannot exist independently without a session. Their existence is entirely dependent on the parent SESSION entity, making this a textbook weak entity relationship.

Decision 3: Using Associative Entities for M:N Relationships

Rationale: Rather than allowing direct many-to-many relationships, we created:

- **ENROLLMENT:** Captures when students enroll in subjects with start/end dates
- **SESSION_ATTENDANCE:** Records actual participation with check-in times
- **TUTOR_SUBJECT:** Tracks qualifications and certification dates

This approach enables us to store relationship-specific attributes that wouldn't fit in either parent entity.

Decision 4: Storing Money as Cents (INTEGER)

Rationale: Using price_cents and amount_cents as integers avoids floating-point precision errors common in financial calculations. \$249.00 is stored as 24900 cents.

Decision 5: Soft Deletion with 'active' Flags

Rationale: Rather than hard-deleting records (which breaks referential integrity and loses historical data), we use active boolean flags. This preserves audit trails while hiding inactive records from daily operations.

1.6 Business Rules Encoded in the Database

Our database enforces critical business constraints at the schema level:

1. **Every package must include at least one session:** CHECK (sessions_included > 0)
2. **Sessions cannot end before they start:** CHECK (scheduled_end > scheduled_start)
3. **Students must have a valid enrollment to attend sessions:** Foreign key constraints
4. **Payment amounts cannot be negative:** CHECK (amount_cents >= 0)
5. **Each student-subject-center-date combination is unique:** UNIQUE (student_id, center_id, subject_id, start_date)
6. **One progress report per student per session:** UNIQUE (session_id, enrollment_id)

Section 2: Database Implementation

Total Number of Tables and Key Relationships:

Our Mathnasium database contains 14 fully implemented tables based on the ERD and data dictionary:

1. LOCATION
2. CENTER
3. TUTOR
4. SUBJECT

5. TUTOR_SUBJECT
6. STUDENT
7. PARENT_GUARDIAN
8. STUDENT_PARENT
9. PACKAGE_PLAN
10. PAYMENT
11. ENROLLMENT
12. SESSION
13. SESSION_ATTENDANCE
14. PROGRESS_REPORT

Key relationships implemented:

- One-to-Many
 - Location → Center
 - Center → Tutor
 - Center → Student
 - Student → Payment
 - Session → Progress_Report
 - Enrollment → Session_Attendance
 - Many-to-Many
 - Tutor ↔ Subject (via TUTOR_SUBJECT)
 - Student ↔ Parent (via STUDENT_PARENT)
- Referential Integrity Enforcement (examples):

- FOREIGN KEY (center_id) REFERENCES center(center_id)
- FOREIGN KEY (student_id) REFERENCES student(student_id)
- All foreign keys validated and tested in Phase 4 inserts.

Representative CREATE TABLE Samples

STUDENT Table

```
CREATE TABLE student (
  student_id BIGINT AUTO_INCREMENT PRIMARY KEY,
  center_id BIGINT NOT NULL,
  name VARCHAR(120) NOT NULL,
  dob DATE,
  email VARCHAR(120),
  phone VARCHAR(20),
  active BOOLEAN NOT NULL DEFAULT TRUE,
  FOREIGN KEY (center_id) REFERENCES center(center_id)
);
```

SESSION Table

```
CREATE TABLE session (
  session_id BIGINT AUTO_INCREMENT PRIMARY KEY,
  center_id BIGINT NOT NULL,
  tutor_id BIGINT NOT NULL,
  subject_id BIGINT NOT NULL,
  scheduled_start TIMESTAMP NOT NULL,
  scheduled_end TIMESTAMP NOT NULL,
  room VARCHAR(40),
  status VARCHAR(20) NOT NULL DEFAULT 'scheduled',
  CHECK (scheduled_end > scheduled_start),
  FOREIGN KEY (tutor_id) REFERENCES tutor(tutor_id),
  FOREIGN KEY (center_id) REFERENCES center(center_id),
  FOREIGN KEY (subject_id) REFERENCES subject(subject_id)
);
```

PACKAGE_PLAN Table

```
CREATE TABLE package_plan (
  plan_id BIGINT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL UNIQUE,
  sessions_included INT NOT NULL CHECK (sessions_included > 0),
  price_cents INT NOT NULL CHECK (price_cents >= 0),
```

```
valid_days INT,  
active BOOLEAN NOT NULL DEFAULT TRUE  
);
```

Representative INSERT statements

Insert Example: STUDENT

```
INSERT INTO student (student_id, center_id, name, dob, email, phone, active) VALUES  
(1, 1, 'Aaron Rivera', '2013-04-12', 'aaron.rivera@school.com', '718-900-1001', TRUE),  
(2, 1, 'Adam Summers', '2012-02-20', 'adam.summers@school.com', '718-900-1002', TRUE);
```

Insert Example: SESSION

```
INSERT INTO session (session_id, center_id, tutor_id, subject_id, scheduled_start, scheduled_end, room,  
status)  
VALUES  
(1, 1, 1, 1, '2024-03-01 15:00', '2024-03-01 16:00', 'A1', 'scheduled');
```

Insert Example: PAYMENT

```
INSERT INTO payment (payment_id, student_id, plan_id, amount_cents, method, paid_at, reference)  
VALUES  
(1, 1, 1, 24900, 'credit_card', CURRENT_TIMESTAMP, 'PMT10001')
```

Changes Made After Phase 4 Testing and Improvement

Foreign Key Creation Order Required Adjustment

- Issue: Tables with dependencies failed to create because referenced tables did not exist yet.
- Fix: Reordered table creation sequence (e.g., LOCATION → CENTER → TUTOR → STUDENT).
- Improvement: No foreign key errors and consistent relational integrity.

CHECK Constraints Updated to Match MySQL Compatibility

- Issue: MySQL initially rejected CHECK constraints during early testing.
- Fix: Adjusted constraint syntax and values (e.g., replacing Unicode characters, enforcing session duration rule).
- Improvement: Ensured valid business logic (no zero-session plans, no negative values, no backwards session times).

UNIQUE Constraints Added for Data Validation

Examples added in Phase 4:

- UNIQUE(student_id, parent_id) in STUDENT_PARENT
- UNIQUE(tutor_id, subject_id) in TUTOR_SUBJECT
- UNIQUE(session_id, enrollment_id) in PROGRESS_REPORT

Improvement: Prevents duplicate or conflicting relationships, supporting cleaner analytics and preventing data corruption.

Standardized Data Types Across Tables

Example: All primary keys converted to BIGINT AUTO_INCREMENT for consistency.

Improvement: Ensured scalability and simplified future joins.

Constraint Validation Summary

1. Foreign Key Validation

Attempting to insert a session with a non-existent tutor_id:

```
INSERT INTO session (center_id, tutor_id, subject_id, scheduled_start, scheduled_end)
VALUES (1, 999, 1, '2024-03-10 10:00', '2024-03-10 11:00');
```

Result: MySQL rejected the insert which is correct behavior.

2. CHECK Constraint Validation

Example: Testing invalid package plan:

```
INSERT INTO package_plan (name, sessions_included, price_cents)
VALUES ('Invalid Plan', 0, 10000);
```

Result: Error due to CHECK (sessions_included > 0).

3. UNIQUE Constraint Validation

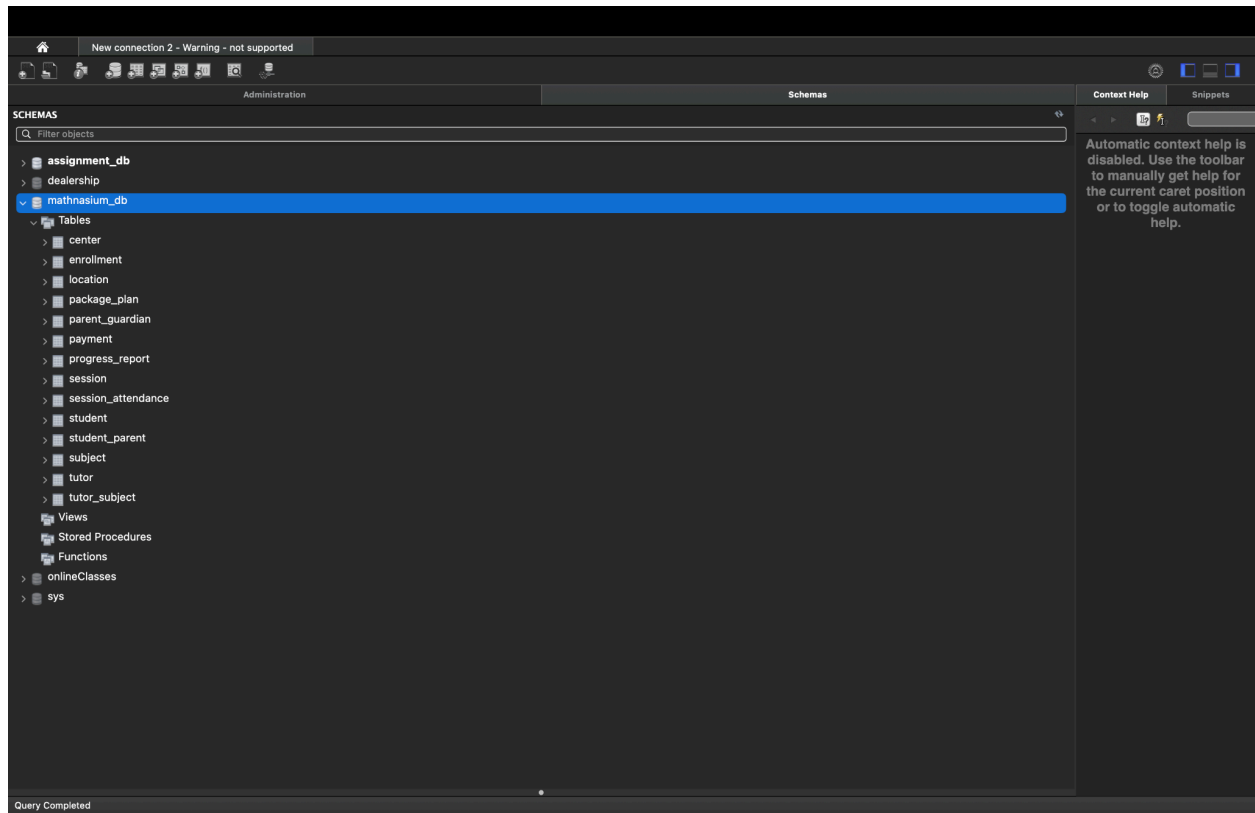
Testing duplicate parent-child relationship:

```
INSERT INTO student_parent (student_id, parent_id, is_primary)
VALUES (1, 1, TRUE);
```

Result: UNIQUE constraint prevents duplicate entries.

Schema and Sample Output Screenshots

Schema Panel



DESCRIBE table statements

1 `DESCRIBE student;`

100% 18:1

Result Grid Filter Rows: Search Export:

	Field	Type	Null	Key	Default	Extra
	student_id	bigint	NO	PRI	<code>NULL</code>	auto_increment
	center_id	bigint	NO	MUL	<code>NULL</code>	
	name	varchar(120)	NO		<code>NULL</code>	
	dob	date	YES		<code>NULL</code>	
	email	varchar(120)	YES		<code>NULL</code>	
	phone	varchar(20)	YES		<code>NULL</code>	
	active	tinyint(1)	NO		1	

```

1 • DESCRIBE student;
2 • DESCRIBE session;

```

100% 18:2

Result Grid Filter Rows: Search Export:

	Field	Type	Null	Key	Default	Extra
	session_id	bigint	NO	PRI	NULL	auto_increment
	center_id	bigint	NO	MUL	NULL	
	tutor_id	bigint	NO	MUL	NULL	
	subject_id	bigint	NO	MUL	NULL	
	scheduled_start	timestamp	NO		NULL	
	scheduled_end	timestamp	NO		NULL	
	actual_start	timestamp	YES		NULL	
	actual_end	timestamp	YES		NULL	
	room	varchar(40)	YES		NULL	
	status	varchar(20)	NO		scheduled	

```

1 • DESCRIBE student;
2 • DESCRIBE session;
3 • DESCRIBE payment;
4

```

100% 18:3

Result Grid Filter Rows: Search Export:

	Field	Type	Null	Key	Default	Extra
	payment_id	bigint	NO	PRI	NULL	auto_increment
	student_id	bigint	NO	MUL	NULL	
	plan_id	bigint	NO	MUL	NULL	
	amount_cents	int	NO		NULL	
	method	varchar(30)	NO		NULL	
	paid_at	timestamp	NO		CURRENT_TIMESTAMP	DEFAULT_GENERATED
	reference	varchar(60)	YES	UNI	NULL	

Failed INSERT (constraint test)

New connection 2 - Warning - not supported

Administration Schemas SQL File 13* SQL File 14* SQL File 15* SQL File 16* SQL File 45* SQL File 46* Context Help Snippets

SCHEMAS Filter objects

- assignment_db
- dealership
- mathnasium_db
 - Tables
 - center
 - enrollment
 - location
 - package_plan
 - parent_guardian
 - payment
 - progress_report
 - session
 - session_attendance
 - student
 - student_parent
 - subject
 - tutor
 - tutor_subject
 - Views
 - Stored Procedures
 - Functions
 - onlineClasses
 - sys

1 • SELECT * FROM student LIMIT 10;

2 • SELECT session_id, tutor_id, subject_id, scheduled_start, room, status

3 FROM session

4 LIMIT 5;

5

100% 1:5

Result Grid Filter Rows: Search Edit: Export/Import: Fetch rows:

session_id	tutor_id	subject_id	scheduled_start	room	status
1	1	1	2024-03-01 15:00:00	A1	scheduled
2	2	2	2024-03-02 16:00:00	B1	scheduled
3	3	3	2024-03-03 17:00:00	C1	scheduled
4	1	1	2024-03-04 15:00:00	A1	scheduled
5	2	2	2024-03-05 16:00:00	B1	scheduled
NULL	NULL	NULL	NULL	NULL	NULL

session 2 Apply Revert

Query Completed

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

New connection 2 - Warning - not supported

Administration Schemas SQL File 13* SQL File 14* SQL File 15* SQL File 16* SQL File 45* SQL File 46* Context Help Snippets

SCHEMAS Filter objects

- assignment_db
- dealership
- mathnasium_db
 - Tables
 - center
 - enrollment
 - location
 - package_plan
 - parent_guardian
 - payment
 - progress_report
 - session
 - session_attendance
 - student
 - student_parent
 - subject
 - tutor
 - tutor_subject
 - Views
 - Stored Procedures
 - Functions
 - onlineClasses
 - sys

2 • SELECT session_id, tutor_id, subject_id, scheduled_start, room, status

3 FROM session

4 LIMIT 5;

5 • SELECT payment_id, student_id, plan_id, amount_cents, method, paid_at

6 FROM payment

7 LIMIT 5;

8

100% 9:7

Result Grid Filter Rows: Search Edit: Export/Import: Fetch rows:

payment_id	student_id	plan_id	amount_cents	method	paid_at
1	1	1	24900	credit_card	2025-11-24 00:31:55
2	2	2	19900	credit_card	2025-11-24 00:31:55
3	3	1	24900	cash	2025-11-24 00:31:55
4	4	1	24900	credit_card	2025-11-24 00:31:55
5	5	2	19900	credit_card	2025-11-24 00:31:55
NULL	NULL	NULL	NULL	NULL	NULL

payment 3 Apply Revert

Query Completed

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

New connection 2 - Warning - not supported

Administration Schemas

SQL File 13* SQL File 14* SQL File 15* SQL File 16* SQL File 45* SQL File 46*

Limit to 1000 rows

Context Help Snippets

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

SCHEMAS

Filter objects

- assignment_db
- dealership
- mathnasium_db
 - Tables
 - center
 - enrollment
 - location
 - package_plan
 - parent_guardian
 - payment
 - progress_report
 - session
 - session_attendance
 - student
 - student_parent
 - subject
 - tutor
 - tutor_subject
 - Views
 - Stored Procedures
 - Functions
 - onlineClasses
 - sys

```

5 SELECT payment_id, student_id, plan_id, amount_cents, method, paid_at
6 FROM payment
7 LIMIT 5;
8 SELECT enrollment_id, student_id, subject_id, start_date, status
9 FROM enrollment
10 LIMIT 5;
11

```

100% 9/10

Result Grid

enrollment_id	student_id	subject_id	start_date	status
1	1	1	2024-01-10	active
2	2	2	2024-01-12	active
3	3	1	2024-01-14	active
4	4	3	2024-01-15	active
5	5	2	2024-01-20	active
NULL	NULL	NULL	NULL	NULL

enrollment 4

Apply Revert

Query Completed

New connection 2 - Warning - not supported

Administration Schemas

SQL File 13* SQL File 14* SQL File 15* SQL File 16* SQL File 45* SQL File 46*

Limit to 1000 rows

Context Help Snippets

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

SCHEMAS

Filter objects

- assignment_db
- dealership
- mathnasium_db
 - Tables
 - center
 - enrollment
 - location
 - package_plan
 - parent_guardian
 - payment
 - progress_report
 - session
 - session_attendance
 - student
 - student_parent
 - subject
 - tutor
 - tutor_subject
 - Views
 - Stored Procedures
 - Functions
 - onlineClasses
 - sys

```

6 FROM payment
7 LIMIT 5;
8 SELECT enrollment_id, student_id, subject_id, start_date, status
9 FROM enrollment
10 LIMIT 5;
11 SELECT * FROM subject;
12

```

100% 23/11

Result Grid

subject_id	name	grade_band	active
1	Elementary Math	Grades 1-5	1
2	Pre-Algebra	Grades 6-7	1
3	Algebra I	Grades 8-9	1
NULL	NULL	NULL	NULL

subject 6

Apply Revert

Query Completed

Section 3 & 4: Use Case Demonstration + Testing/Validation

3.1 REGISTER A NEW STUDENT

Business goal: A front-desk staff member registers a new student, links a parent/guardian, and records a package purchase so the student can start attending sessions.

Step 1 – Add a new parent/guardian

User action: Staff opens “Add Parent/Guardian” form and fills in contact details.

SQL:

```
INSERT INTO parent_guardian (name, email, phone, relationship_to_student)
VALUES ('Nadia Ortiz', 'nadia.ortiz@family.com', '718-555-6101', 'Mother');
```

Output:

	parent_id	name	email	phone	relationship_to_student
►	1	Maria Rivera	maria.rivera@family.com	718-555-6001	Mother
	2	Daniel Summers	daniel.summers@family.com	718-555-6002	Father
	3	Sofia Romero	sofia.romero@family.com	718-555-6003	Mother
	4	Laura Bryson	laura.bryson@family.com	718-555-6004	Mother
	5	Imran Khalil	imran.khalil@family.com	718-555-6005	Father
	6	Karen Lane	karen.lane@family.com	718-555-6006	Mother
	7	Michael Daniels	michael.daniels@family.com	718-555-6007	Father
	8	Lily Chen	lily.chen@family.com	718-555-6008	Mother
	9	Grace Chang	grace.chang@family.com	718-555-6009	Mother
	10	Samuel Barnes	samuel.barnes@family.com	718-555-6010	Father
	11	Nadia Ortiz	nadia.ortiz@family.com	718-555-6101	Mother

Step 2 – Add a new student at the Fresh Meadows center

User action: Staff opens “Add Student” form and enters student info, selecting the center “Fresh Meadows Math Center”.

SQL:

```
INSERT INTO student (student_id, center_id, name, dob, email, phone, active)
VALUES (11, 1, 'Brandon Ortiz', '2014-11-23', 'brandon.ortiz@school.com', '718-900-1011', TRUE);
```

Output:

	student_id	center_id	name	dob	email	phone	active
▶	1	1	Aaron Rivera	2013-04-12	aaron.rivera@school.com	718-900-1001	1
	2	1	Adam Summers	2012-02-20	adam.summers@school.com	718-900-1002	1
	3	1	Alyssa Romero	2014-09-05	alyssa.romero@school.com	718-900-1003	1
	4	1	Ava Bryson	2013-06-18	ava.bryson@school.com	718-900-1004	1
	5	1	Aiden Khalil	2011-11-02	aiden.khalil@school.com	718-900-1005	1
	6	1	Allison Lane	2013-12-03	allison.lane@school.com	718-900-1006	1
	7	1	Andrew Daniels	2012-08-22	andrew.daniels@school.com	718-900-1007	1
	8	1	Chloe Chen	2014-01-15	chloe.chen@school.com	718-900-1008	1
	9	1	Claire Chang	2013-07-10	claire.chang@school.com	718-900-1009	1
	10	1	Caleb Barnes	2011-10-28	caleb.barnes@school.com	718-900-1010	1
	11	1	Brandon Ortiz	2014-11-23	brandon.ortiz@school.com	718-900-1011	1

Step 3 – Record package purchase

User action: Staff selects “Monthly Unlimited 8” as the plan and records payment by credit card.

SQL:

```
INSERT INTO payment (student_id, plan_id, amount_cents, method, reference)
VALUES (11, 1, 24900, 'credit_card', 'PMT10011');
```

Output:

	payment_id	student_id	plan_id	amount_cents	method	paid_at	reference
▶	1	1	1	24900	credit_card	2025-11-24 22:17:44	PMT10001
	2	2	2	19900	credit_card	2025-11-24 22:17:44	PMT10002
	3	3	1	24900	cash	2025-11-24 22:17:44	PMT10003
	4	4	1	24900	credit_card	2025-11-24 22:17:44	PMT10004
	5	5	2	19900	credit_card	2025-11-24 22:17:44	PMT10005
	6	6	1	24900	debit_card	2025-11-24 22:17:44	PMT10006
	7	7	2	19900	cash	2025-11-24 22:17:44	PMT10007
	8	8	1	24900	credit_card	2025-11-24 22:17:44	PMT10008
	9	9	2	19900	credit_card	2025-11-24 22:17:44	PMT10009
	10	10	1	24900	debit_card	2025-11-24 22:17:44	PMT10010
	11	11	1	24900	credit_card	2025-12-08 14:54:34	PMT10011

3.2 CENTER ANALYTICS

Business goal: The center director wants a quick dashboard showing (a) revenue by plan for a period, (b) current active enrollments by subject, and (c) upcoming sessions per tutor.

3.1 Revenue by package plan for a given month

User action: Director selects “Month: March 2024” in the revenue report.

SQL:

```

SELECT
  pp.name AS plan_name,
  COUNT(pay.payment_id) AS num_payments,
  SUM(pay.amount_cents) / 100.0 AS total_revenue_usd
FROM payment pay
JOIN package_plan pp
  ON pay.plan_id = pp.plan_id
WHERE pay.paid_at >= '2025-11-01' AND pay.paid_at < '2025-12-01'
GROUP BY pp.name
ORDER BY total_revenue_usd DESC;

```

Output:

	plan_name	num_payments	total_revenue_usd
▶	Monthly Unlimited 8	6	1494.0000
	Flexible 5-Pack	4	796.0000

4.1 TESTING & VALIDATION

After creating the schema and loading the sample data, I validated the database in four ways:

1. Structural checks
 - Verified all tables were created:
 - Verified key columns and constraints:


```

SHOW CREATE TABLE student;
SHOW CREATE TABLE session;
SHOW CREATE TABLE session_attendance;

```
2. Basic data sanity checks
 - Confirmed row counts matched my inserts:


```

SELECT COUNT(*) FROM student;
SELECT COUNT(*) FROM tutor;
SELECT COUNT(*) FROM enrollment;
SELECT COUNT(*) FROM session;

```
3. Relationship checks
 - Ran joins across foreign keys to ensure parent-child relationships are consistent
4. Constraint enforcement tests
 - Manually attempted inserts/updates that violate:
 - NOT NULL
 - UNIQUE
 - CHECK
 - FOREIGN KEY

SECTION 5: FUTURE ENHANCEMENTS & RECOMMENDATIONS

User Interface & Workflow Improvements

Enhancement 1: Web-Based Parent Portal

Problem: Parents currently have no direct visibility into their child's progress and attendance. They must call the center or wait for monthly reports.

Proposed Solution: Build a secure web portal where parents can:

- View real-time attendance records with check-in/check-out times
- Read tutor progress reports immediately after each session
- See upcoming scheduled sessions
- Download payment receipts and transaction history
- Update contact information

Database Impact:

new portal_user table with authentication credentials

- Add last_accessed timestamp to track engagement
- Create parent_message table for center-to-parent communication
- Add receipt_requested boolean to payment table

```
CREATE TABLE portal_user (  
  user_id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  parent_id BIGINT NOT NULL,  
  email VARCHAR(120) NOT NULL UNIQUE,  
  password_hash VARCHAR(255) NOT NULL,  
  last_login TIMESTAMP NULL,  
  account_status VARCHAR(20) NOT NULL DEFAULT 'active',  
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (parent_id) REFERENCES parent_guardian(parent_id)  
);
```

```
CREATE TABLE parent_message (  
  message_id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  parent_id BIGINT NOT NULL,  
  subject VARCHAR(200) NOT NULL,  
  body TEXT NOT NULL,  
  sent_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  read_at TIMESTAMP NULL,  
  FOREIGN KEY (parent_id) REFERENCES parent_guardian(parent_id)  
);
```

Business Benefits:

- **Improved Communication:** Reduces phone calls and administrative burden
- **Parent Satisfaction:** Transparency builds trust and retention
- **Payment Collection:** Online access to receipts simplifies tax documentation
- **Engagement Metrics:** Track which parents actively monitor student progress

Data integrity or performance

We suggest adding indexes to commonly accessed foreign key columns (such as student_id, tutor_id, and subject_id) to improve efficiency and preserve data integrity as the Mathnasium system grows. As data volume grows, this will avoid full-table scans, guarantee quick recovery of attendance and progress information, and shorten query execution times for JOIN operations. By putting indexes in place, the system is better equipped to handle increased workloads and improves the effectiveness of analytical tools. Here are some examples of index commands:

```
CREATE INDEX idx_enrollment_student  
ON enrollment(student_id);
```

```
CREATE INDEX idx_session_tutor  
ON session(tutor_id);
```

```
CREATE INDEX idx_progress_enrollment  
ON progress_report(enrollment_id);
```

Why This Enhances Our System:

- Quicker lookups and JOIN queries
- Increased scalability with the addition of session, student, and payment records
- Phase 5 analytic query execution time reduction
- Increased referential integrity as a result of reliable foreign key performance

New features or automation

Auto-Generate Monthly Revenue Report

Purpose: A director no longer needs to run SQL manually.

Automation: Create a stored procedure that compiles monthly revenue and stores it in a monthly_reports table.

SQL:

```
CREATE TABLE monthly_reports (  
    report_month DATE PRIMARY KEY,  
    total_revenue DECIMAL(10,2),  
    last_generated TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE PROCEDURE generate_monthly_revenue(IN report_month DATE)  
BEGIN  
    INSERT INTO monthly_reports (report_month, total_revenue)  
    SELECT report_month,  
           SUM(amount_cents)/100.0  
    FROM payment  
    WHERE DATE_FORMAT(paid_at, '%Y-%m') = DATE_FORMAT(report_month, '%Y-%m');  
END;
```

Contribution

Sadia Sultana:

For Phase 5, I was responsible for writing Section 1 (System Overview & Project Evolution) and Section 5.1 (User Interface and Workflow Improvements). In Section 1, I explained our database project's purpose, documented our ER diagram, and described how our design evolved throughout the semester. For Section 5.1, I proposed three user-focused enhancements: a web-based parent portal for real-time attendance and progress tracking, mobile check-in kiosks to streamline the check-in process, and automated email campaigns for family engagement. Each enhancement included problem statements, proposed solutions, user workflow designs, and ROI calculations showing business impact. Throughout the project, our team communicated using Zoom for weekly meetings and iMessage for daily coordination and quick questions, which helped us stay organized and ensure all our sections connected properly.

Shifath Hossain:

I developed and deployed the entire Mathnasium database system, along with the entire relational schema, all constraints and realistic sample data. I developed user end-to-end workflows: student registration, session scheduling and attendance, and center analytics and illustrated each workflow by the exact SQL queries that would be run by the interface and their results. I tested the correctness of the schema using targeted testing, such as the successful execution of operations and the explicit failure used to verify the existence of foreign key, unique, check, and not-null constraints that were implemented in a proper manner. Lastly, I suggested and developed a number of automation options, such as generating monthly

revenues, controlling the number of sessions, and expiring packages. In general, I was involved in the entire design, implementation, testing, and validation of a functional and operationally realistic database system in a Mathnasium learning center.

Avneet Kaur

This phase required me to analyze our entire schema and record the database's implementation after Phase 4. I chose typical CREATE and INSERT fragments, highlighted important table relationships, and described the structural changes we made following testing. Additionally, I produced a failed INSERT test to show referential integrity, ran DESCRIBE statements, and produced sample output screenshots to check our constraints. Moreover, I helped confirm that the implemented schema aligns with our ER model and supports accurate, efficient data retrieval for the use cases demonstrated in Phase 5. My efforts made sure that our final documentation appropriately depicts a well-designed and operational Mathnasium database.